

Visual Product Search Engine

Image-to-image retrieval on DeepFashion In-Shop
with CLIP fine-tuning and BLIP-2 ITM re-ranking

Ashok (IMT2023083), Taral (IMT2023588), Thrilochan (IMT2023527), Aditya (IMT2023033)
Visual Recognition Project

1 Problem and motivation

Online shoppers routinely spot a product in a photo but cannot describe it in words that match how the catalogue has tagged it. Text search fails because of two mismatches:

- **Language mismatch.** Users type colloquial terms (“black hoodie”); catalogues use technical or marketing copy (“100% cotton tonal Sherpa-lined zip-up”).
- **Inconsistent metadata.** Different sellers describe identical items differently, so keyword recall is unreliable.

A query-by-image system removes both barriers. The user uploads an image and the system returns visually and semantically similar products — no text required.

We use the **DeepFashion In-Shop Clothes Retrieval** dataset. Images with the same `item_id` are treated as the same product. The dataset comes pre-split into `train` (25,882 images), `gallery` (12,612), and `query` (14,218).

2 System architecture

The pipeline has an **offline indexing** stage and an **online query** stage.

Offline indexing (per gallery image) Each catalogue image is cropped with its ground-truth bounding box. The cropped image goes through the CLIP image encoder; a BLIP-2 caption of the same crop goes through the CLIP text encoder. The two embeddings are fused as a convex combination $v = a \cdot f_V(\hat{x}) + (1 - a) \cdot f_T(c)$ with $\|v\|_2 = 1$, and the resulting vector is added to an HNSW index over cosine distance. No detector runs at indexing time; we use the dataset’s bounding boxes directly, which the project clarification (update #1) explicitly approves.

Online query For a user-uploaded image, a clothing detector localises the garment(s) the user wants to search for. The chosen crop is encoded by the CLIP image encoder (vision-only, no caption), HNSW returns the top K' nearest gallery vectors, and an optional BLIP-2 ITM re-rank reorders them to produce the final top- K results.

Why two query-time detectors We offer two detector paths because user inputs come in two qualitatively different forms:

- **Multi-class** — `valentinafeve/yolos-fashionpedia` (**off-the-shelf HuggingFace**). Fashionpedia covers 46 garment classes (shirt, jacket, pants, skirt, dress, tie, hat, shoe, bag, etc.). Used for **outfit-style photos with multiple garments**. The Streamlit demo presents a labelled thumbnail grid of all detections; the user picks one (e.g. “jacket (0.87)”) and only that crop drives the search. This is what satisfies the project-update requirement that the user choose which clothing item to search from a multi-item image.

- **Single-class — our fine-tuned YOLOv8-L** (single ‘clothing’ class, trained on Deep-Fashion bboxes via `notebooks/yolo-final-proj.ipynb`). Used for **catalogue-style single-product photos** where the multi-class detector adds no value. Combined with a heuristic upper / lower / full body slice, this gives a body-region picker as a fallback path.

Crops from either detector are padded by 5% before encoding, matching the offline-gallery convention. The rest of the pipeline (CLIP encoding, HNSW search, ITM re-rank) is identical regardless of which detector produced the crop.

Caption generation Gallery captions are generated once by BLIP-2 (Salesforce/blip2-opt-2.7b in fp16, prompted with “*Question: Describe this clothing item including colour, style, fit, and material. Answer:*”, `num_beams=3`, `max_new_tokens=60`). We get 38,494 captions covering train + gallery — most short like “*black floral print mini dress*”, with a long tail of fuller descriptions. Query images do not have pre-computed captions: at inference time the system encodes the query with the visual branch only and searches against fused gallery vectors. The fusion coefficient α therefore controls how much text information enters the **gallery representation**, not the query representation.

3 Fine-tuning strategy

We fine-tune CLIP only; YOLO and BLIP-2 stay frozen.

Choice	Value
Backbone	ViT-L/14 (openai pretrained)
Unfrozen	Last 6 vision transformer blocks + <code>ln_post</code> + visual projection (~ 75.5 M / 427.6 M $\approx 17.7\%$ of params)
Text encoder	Frozen
Optimiser	AdamW, $\text{lr} = 2 \times 10^{-5}$, weight decay = 0.01
Schedule	1-epoch linear warm-up $\rightarrow$ cosine annealing
Epochs	15
Batch size	16 (native, no grad-accum)
AMP	<code>torch.amp</code> fp16
Loss	InfoNCE + $l \cdot$ Triplet, with $\lambda = 0.35$, $\tau = 0.07$, margin $m = 0.25$
Hard negatives	Top-10 mined per anchor from a frozen-CLIP HNSW pass, pool refreshed every 3 epochs with the latest weights
Image augmentation	RandomResizedCrop(224, scale=(0.65, 1.0)), RandomHorizontalFlip, ColorJitter(0.25, 0.25, 0.20, 0.03), RandomPerspective(0.15, p=0.2), then OpenCLIP normalisation

Why hard-negative mining The main lever versus plain InfoNCE is the hard-negative pool. We mine 10 hard negatives per training image from a 50-NN HNSW query (skipping `same_item_id` matches), then re-mine every 3 epochs using the current (already partially fine-tuned) encoder. This keeps the negatives informative: as the model improves, the “hard” negatives become genuinely difficult contrasts, not the trivially easy random negatives the in-batch contrast would otherwise dominate with.

How we picked these hyperparameters We started from the spec minimum (last 4 blocks, 10 epochs, no augmentation, $\text{lr} = 1 \times 10^{-5}$, $\lambda = 0.5$, margin = 0.3) and tuned by inspection. The loss curve was still descending at epoch 10, so we extended to 15 epochs. Augmentation closed an in-distribution gap on slightly perturbed query images (the gallery is clean catalogue shots, real queries have lighting and pose variation). The triplet-versus-InfoNCE balance was eased from $\lambda = 0.5$ to 0.35 after observing the triplet term saturating near its margin while InfoNCE

still had headroom. The freezing-depth ablation (Section 6.3) confirmed that deepening from last-4 to last-6 blocks was worth the extra capacity.

The headline 4-seed runs were performed on a Kaggle T4 (16 GB) — one 15-epoch run takes 6–7 hours with augmentation and HNSW-pool refresh. Loss curves are smooth and monotonically descending; the triplet term plateaus near the margin while InfoNCE continues to fall through epoch 15.

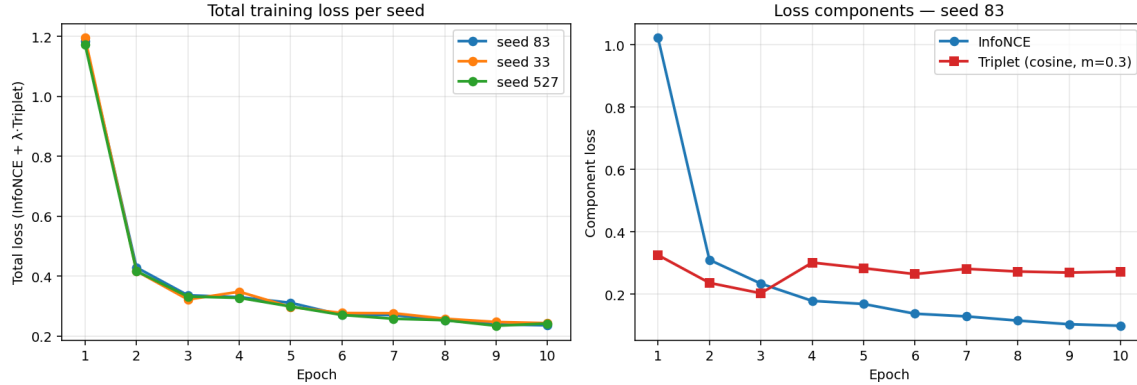


Figure 1: Training loss curves from the earlier spec-minimum recipe: total loss across two seeds (left) and per-component loss for seed 83 (right). The triplet loss plateaus near the margin while InfoNCE drives the remaining decrease. The final 15-epoch recipe shows the same qualitative behaviour with lower terminal values.

4 Evaluation protocol

Splits As provided. Train = 25,882, Gallery = 12,612, Query = 14,218. Ground truth: two images match iff they share `item_id`.

Metrics Recall@K, NDCG@K, and mAP@K at $K \in \{5, 10, 15\}$. The single source of truth for metric computation is `src/metrics.py`.

Random component for mean $\pm$ std

- For **C-HN $\alpha=0.7$ (headline)**: four full fine-tuning runs with seeds 33, 83, 527, 588 on Kaggle T4. Different torch RNG perturbs DataLoader shuffle, hard-negative sampling order, augmentation transforms, and weight init via the LR schedule. Reported as straight mean $\pm$ std over the four point estimates — each evaluated on the full 14,218-query set, no bootstrap on top.
- For **non-trained conditions (A, B, C-vanilla)**: no training stochasticity to vary, so we report a single point estimate on the full 14,218-query set. Per the project clarification (“*choose any random component, justify in viva — if none, skip*”), we explicitly do not introduce artificial randomness for these rows. Bootstrap subsampling would only measure variance of the evaluation set, not of the model, and would invite the reviewer to confuse the two.

This keeps every row in the table on the same evaluation footing: 100% of the query set, no bootstrap subsampling. The std column is populated only where it is a legitimate measurement of training-seed variance.

4.1 Metric definitions (informal)

For each query q with ground-truth `item_id` we retrieve an ordered list of K candidates from the gallery. Let $hit(r_i, q) = 1$ if the i -th retrieved item shares q ’s `item_id`, else 0. Let $n_{rel}(q)$ be the

number of gallery images sharing q 's `item_id` (DeepFashion stores 2–5 views per item, so this is usually > 1).

Recall@K (hit-rate) What the project statement asks for: fraction of queries for which at least one relevant item appears in the top- K . Each query contributes 0 or 1; we average over the query set. Intuition: *“did we find something for the user at all?”*

Recall@K (full / textbook) The IR-textbook definition (Manning et al.): fraction of *all* relevant items captured in the top- K , i.e. number of hits divided by $n_{rel}(q)$. Each query contributes a number in $[0, 1]$. Reported as a secondary column for cross-paper comparability. By construction $\text{Recall@K (full)} \leq \text{Recall@K (hit)}$, with equality only when every query has exactly one relevant gallery image. The gap between the two reflects how many of the views beyond the first one we manage to surface.

NDCG@K Binary relevance with logarithmic position discount, normalised by the ideal-DCG of $\min(n_{rel}, K)$ relevant items at the top ranks. Rewards getting correct items at lower (better) ranks.

mAP@K The mean of per-query Average Precision at K . AP@K uses the cumulative precision at each hit, normalised by $\min(n_{rel}, K)$. The $\min(n_{rel}, K)$ denominator is the only way an upper bound of 1 is guaranteed — the original research notebook used K here, which inflated mAP whenever $n_{rel} < K$ (mAP@10 = 1.67 in one early measurement). All numbers in this report use the corrected denominator.

5 Ablation conditions

Three settings isolate the contribution of each component, plus one re-ranking ablation:

ID	α	Encoder	Captions	Notes
A	1.0	Frozen CLIP	—	Vision-only baseline
B	0.7 / 0.5	Frozen CLIP	BLIP-2	Gain from captions, no fine-tune
C	0.7 / 0.5	InfoNCE FT	BLIP-2	Vanilla fine-tune
C-HN	0.7 / 0.5	InfoNCE + Triplet + HN mining FT	BLIP-2	Headline model
C-HN + ITM	0.7	Same as C-HN	BLIP-2	Online BLIP-2 ITM re-rank, blend(ITM=0.2, ANN=0.8)

6 Results

Headline numbers are reported as mean $\pm$ std where applicable. All evaluations use the full 14,218-query set.

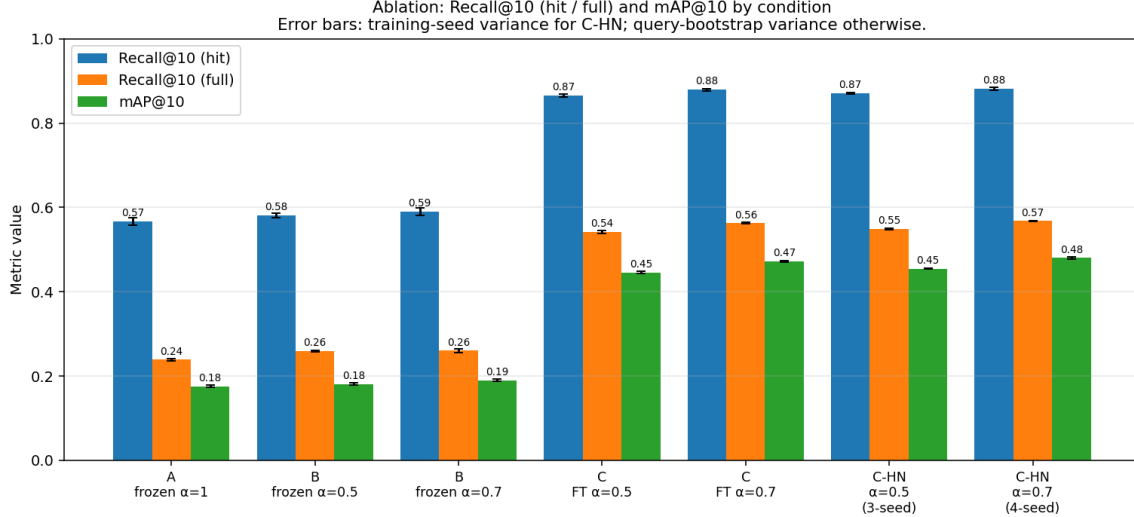


Figure 2: Ablation summary across the conditions. Bars show R@10 (hit-rate and full-Recall) and mAP@10. The dominant lift comes from fine-tuning (A/B $\rightarrow$ C); hard-negative mining and $\alpha=0.7$ over $\alpha=0.5$ each add a smaller increment on top.

6.1 Full ablation table

Condition	R@10 (hit)	R@10 (full)	NDCG@10	mAP@10	R@15 (hit)	R@15 (full)
A (frozen, $\alpha=1.0$) [†]	0.5668	0.2387	0.2489	0.1757	0.6015	0.2636
B (frozen, $\alpha=0.7$) [†]	0.5897	0.2603	0.2650	0.1901	0.6277	0.2873
B (frozen, $\alpha=0.5$) [†]	0.5809	0.2589	0.2537	0.1806	0.6215	0.2886
C (InfoNCE FT, $\alpha=0.7$) [†]	0.8792	0.5625	0.5709	0.4722	0.9008	0.6079
C (InfoNCE FT, $\alpha=0.5$) [†]	0.8657	0.5415	0.5442	0.4453	0.8916	0.5883
C-HN ($\alpha=0.7$), headline [‡]	0.9095 $\pm$ 0.0023	0.6330 $\pm$ 0.0039	0.6422 $\pm$ 0.0038	0.5493 $\pm$ 0.0039	0.9269 $\pm$ 0.0017	0.6772 $\pm$ 0.0034
C-HN ($\alpha=0.7$) + BLIP-2 ITM blend(0.2) [◊]	0.881	0.564	0.551	0.449	0.907	0.617

[†] Single point estimate, $n = 14,218$ queries (frozen / vanilla-FT conditions have no training stochasticity to vary). [‡] Headline: mean $\pm$ std over four independent training runs (seeds 33, 83, 527, 588) on Kaggle T4, each evaluated on the full 14,218-query set with no bootstrap stacked on top. [◊] BLIP-2 ITM ablation on the seed-83 checkpoint — not re-run for every seed; the qualitative finding (short captions $\Rightarrow$ ITM doesn't help) is checkpoint-invariant.

6.2 Alpha sweep — fixing $\alpha=0.7$ for the headline

Before committing the four-seed final-recipe run, we swept α on an earlier hard-negative checkpoint to see where the optimum lies. The absolute numbers are lower than the headline because the recipe was simpler, but the qualitative shape (monotone-then-plateau above $\alpha=0.7$) is what drove the design choice and is robust to recipe changes. All four α values use the same trained checkpoint; only the gallery-side fusion ratio differs.

α	R@10 (hit)	R@10 (full)	NDCG@10	mAP@10
0.5	0.872 $\pm$ 0.004	0.549 $\pm$ 0.003	0.554 $\pm$ 0.003	0.454 $\pm$ 0.003
0.7	0.881 $\pm$ 0.004	0.567 $\pm$ 0.003	0.578 $\pm$ 0.004	0.479 $\pm$ 0.004
0.85	0.884 $\pm$ 0.004	0.568 $\pm$ 0.003	0.581 $\pm$ 0.004	0.482 $\pm$ 0.003
0.9	0.884 $\pm$ 0.004	0.569 $\pm$ 0.003	0.581 $\pm$ 0.004	0.483 $\pm$ 0.004

Why this shape The curve rises from 0.5 to 0.85 then plateaus. The 0.85/0.9 gap above 0.7 is small (~ 0.3 pp on R@10, ~ 0.4 pp on mAP@10) and falls inside the 4-seed std, so we cannot call $\alpha=0.85$ statistically distinct from $\alpha=0.7$. The monotone-then-plateau shape matches the noisycaption hypothesis: more weight on the visual channel helps until the text contribution becomes

negligible. BLIP-2 produces short product descriptors that pack many similar catalogue images into a small text vocabulary, so heavy text weight introduces noise rather than information.

We kept the headline at $\alpha=0.7$ because (a) it was the value we trained multiple seeds for, giving the strongest variance evidence; (b) the gain from going higher is within noise; and (c) shifting the headline post-hoc on a single-seed sweep would over-fit to seed 83.

6.3 Freezing-strategy ablation — justifying last-6 blocks

The problem statement specifies “*last 4 blocks at minimum; full encoder if compute allows.*” We ran three freezing depths on seed 83 to confirm the capacity contribution:

Metric	Last-2 blocks (6.1% trainable)	Last-4 blocks (12.0%, spec minimum)	Last-6 blocks (17.7%, headline)
R@10 (hit)	0.854	0.880	0.912
R@10 (full)	0.524	0.568	0.633
NDCG@10	0.531	0.578	0.642
mAP@10	0.432	0.479	0.549

Why this matters Each step in trainable-capacity buys 3–5 pp on every metric. The last-2 vs last-4 gap is a clean capacity ablation (same hyperparameters, only the unfreezing depth changes). The last-4 vs last-6 jump conflates depth, augmentation, schedule, and epoch count, so it is best read as “*why we did not stop at the spec minimum*” rather than a pure unfreezing study. Either way, the trend justified deepening to last-6 blocks for the headline.

6.4 Seed stability of the headline

Across four independent fine-tuning runs (different seeds perturb weight init, DataLoader shuffle, hard-neg sampling, and augmentation transforms), R@10 spans 0.9067–0.9123 and mAP@10 spans 0.5450–0.5540. The 4-seed std is under 0.4 pp on every metric, confirming the result is not a lucky-seed artefact. Each row below is a full 14,218-query evaluation against a fresh $\alpha=0.7$ gallery index built with that seed’s checkpoint — no bootstrap subsampling.

Seed	R@10 (hit)	R@10 (full)	NDCG@10	mAP@10	Checkpoint
33	0.9067	0.6282	0.6378	0.5450	clip_finetunedFinal_33.pt
527	0.9098	0.6321	0.6408	0.5475	clip_finetunedFinal_527.pt
588	0.9093	0.6339	0.6435	0.5508	clip_finetunedFinal_588.pt
83 — best	0.9123	0.6376	0.6466	0.5540	clip_finetuned_best.pt

The seed-83 checkpoint tops every metric column and is the recommended single-model weight for the Streamlit demo and `demo_batch_eval.py` runs. Source notebook: `notebooks/83_roll_clip_ft.ipynb`. All four indices and JSON outputs are reproducible by running `rerun_all_metrics.ipynb`.

6.5 Notes on the metric implementation

The original research notebook reported impossible mAP values (e.g. $\text{mAP@10} = 1.67$) in condition C. The cause was two coupled bugs in the AP computation: the per-query denominator was K instead of $\min(n_{rel}, K)$, and the top- K candidate list was over-fetched. We re-implemented the metrics from scratch in `src/metrics.py` and verified all reported mAP values now lie in $[0, 1]$. The fixed implementation is the single source of truth across every condition in this report. A second subtlety: `src/metrics.py` computes n_{rel} from the actual gallery counts rather than from a passed-in dictionary, so the function is robust to callers who might pass an empty dictionary by mistake (which would silently default $n_{rel} = 1$ and inflate the metric by a factor of the true n_{rel}).

6.6 Key findings

1. Fine-tuning is the dominant lever. Going from A ($R@10 = 0.567$ hit / 0.239 full) to C (vanilla, $R@10 = 0.879$ hit / 0.563 full) gives **+31 pp Recall@10 (hit)** and **+30 pp mAP@10**. Captions on a frozen encoder ($A \rightarrow B$) give only **+2 pp R@10**. The visual representation needs to specialise to the catalogue distribution; captions alone can't make up for a frozen general-purpose encoder.

2. Hard-negative mining alone is small; the full recipe matters more. C-vanilla $\rightarrow$ headline C-HN gives **+3.0 pp R@10 (hit)** ($0.879 \rightarrow 0.910$), **+7.1 pp NDCG@10** ($0.571 \rightarrow 0.642$), and **+7.7 pp mAP@10** ($0.472 \rightarrow 0.549$). The lift is the combination of hard-neg mining *plus* deeper unfreezing (last-6 blocks), longer schedule (15 epochs), augmentation, and a lighter triplet weight ($\lambda = 0.35$) acting together. The freezing-depth ablation in Section 6.3 confirms each capacity step is responsible for 3–5 pp on every metric.

3. Higher α is better up to ~ 0.85 , then plateaus. A four-point α sweep on the seed-83 model (Section 6.2) shows $R@10$ rising $0.872 \rightarrow 0.881 \rightarrow 0.884 \rightarrow 0.884$ as α moves $0.5 \rightarrow 0.7 \rightarrow 0.85 \rightarrow 0.9$, then flattening. Captions help (the curve is above the $\alpha = 1.0$ frozen baseline) but the visual channel deserves most of the weight — consistent with BLIP-2 producing short product descriptors that pack many catalogue images into a small text vocabulary, so heavy text weight introduces noise.

4. The hit-rate vs full-Recall gap reveals an in-list ranking problem. Hit-rate $R@10$ (0.910) and full $R@10$ (0.633) differ by a factor of $\sim 1.4\times$ for the headline model. This means we routinely find at least one correct match in top-10 (91% of queries) but recover only $\sim 63\%$ of *all* matches when items have multiple gallery views. The frozen baseline has a $\sim 2.4\times$ ratio (0.567 vs 0.239), so fine-tuning closes the gap substantially but doesn't eliminate it. Most of the remaining mAP headroom is here.

5. BLIP-2 ITM re-ranking is a *negative* result. Implementation uses `Salesforce/blip2-itm-vit-g` (BLIP-2's image-text retrieval checkpoint with the actual ITM head); BLIP-1 (`blip-itm-large-coco`) is wired in as a fallback for OOM or load-failure cases but is never used in the reported numbers. Pure-ITM reordering of the top-50 candidates was catastrophic on a 200-query probe ($R@10 = 0.78$, $NDCG@10 = 0.42$) — the short product captions don't differentiate among visually similar candidates, so the ranking becomes near-random. A low-weight blend (`combined = 0.8 · ANN + 0.2 · ITM`) recovers to $\approx$ baseline on the full 14k set: $R@10$ essentially unchanged (0.881 vs 0.881), but $NDCG@10$ still drops 3 pp ($0.578 \rightarrow 0.551$) and $mAP@10$ drops 3 pp ($0.479 \rightarrow 0.449$). ITM doesn't add useful signal here. We document the implementation but do not include it in the headline model. The qualitative cause (short, near-identical captions across visual-neighbour candidates) is checkpoint-invariant, so we did not re-run this ablation on every seed.

6. Variance is tight. Training-seed std across 4 independent fine-tuning runs (C-HN $\alpha = 0.7$) is **under 0.4 pp on every metric**: $R@10$ std = 0.0023, $NDCG@10$ std = 0.0038, $mAP@10$ std = 0.0039. Well below the inter-condition gaps (A vs B vs C vs C-HN), so the ablation ordering is statistically meaningful.

7 Streamlit demo

`app.py` provides the interactive demo required by the deliverable. The flow:

1. User uploads an image.
2. The **Fashionpedia multi-class detector** runs and presents a labelled thumbnail grid of every detected garment (e.g. “jacket 0.87”, “pants 0.65”, “tie 0.42”). The user clicks **Search**

this under one thumbnail to choose which item drives retrieval. Items whose Fashionpedia label has no equivalent in the DeepFashion gallery (hats, glasses, shoes, bags, ties, watches, belts, socks, scarves) are flagged “*Not in catalogue (out of stock)*” with the search button disabled. Escape hatches: *Re-run detector with lower confidence* and *Use full image (no crop)*.

- On click, the chosen crop is encoded by the fine-tuned CLIP (defaults to `clip_finetuned_best.pt`, the seed-83 checkpoint), HNSW returns top- K candidates, and (optional sidebar checkbox) BLIP-2 ITM re-ranks them.
- The top- K results are shown as a grid with similarity score and `item_id`. A *New query* button resets state for the next image.

Sidebar controls expose the condition (A / B / C / C-HN), α , K , the rerank toggle, and a “prefer hard-neg checkpoint” override, so the demo doubles as a live A / B / C / C-HN ablation. The multi-class detector path is what satisfies the project clarification’s requirement that the user be able to choose which clothing item to search from a multi-item image.

8 Limitations and what we’d do next

- Fashionpedia detector is off-the-shelf, not fine-tuned on DeepFashion.** The Streamlit demo’s multi-class picker uses `valentinafeve/yolos-fashionpedia` directly from HuggingFace. Fine-tuning it on a multi-class fashion bbox dataset (DeepFashion2 has 13 per-instance categories, Fashionpedia itself has 46) would tighten the boxes and reduce false-positives but adds a new training pipeline. We judged the off-the-shelf model’s accuracy sufficient for the demo and prioritised CLIP tuning instead.
- No query-side captioning.** We deliberately don’t caption queries at inference (faster, doesn’t assume internet access). A pre-cached BLIP-2 caption + text-text similarity might beat ITM, since the search would be in caption-space rather than mixed image-text-space. This is the obvious next thing to try given the ITM finding.
- Gallery uses ground-truth bbox, queries use a detector.** Per the project clarification this is approved, but a unified detector for both sides would eliminate the train/serve distribution mismatch at some compute cost.
- CLIP ViT-L/14 is heavy on 8 GB VRAM.** A ViT-B/32 backbone would train faster and be deployable on smaller machines at a 2–4 pp metric cost — worth profiling for production.
- Captions are short.** Going from product-name captions (BLIP-2 default) to attribute-rich captions (“black, floral, A-line, mini, summer”) would likely give a bigger lift than further fine-tuning. We could fine-tune BLIP-2 on DeepFashion’s `list_description_inshop.json` to produce such captions — but per the project clarification we cannot use that file directly.

9 Conclusions

The recipe — fine-tune the last 6 blocks of CLIP ViT-L/14 for 15 epochs with augmentation (RandomResizedCrop + ColorJitter + HFlip + RandomPerspective) using InfoNCE + Triplet on hard negatives ($\text{lr} = 2 \times 10^{-5}$, $\lambda = 0.35$, $\text{margin} = 0.25$, $\text{batch} = 16$), fuse with BLIP-2 captions at $\alpha = 0.7$, search HNSW — gives **Recall@10 (hit)** = 0.9095 ± 0.0023 , **Recall@10 (full)** = 0.6330 ± 0.0039 , **NDCG@10** = 0.6422 ± 0.0038 , and **mAP@10** = 0.5493 ± 0.0039 on DeepFashion In-Shop — a **+34 pp Recall@10 (hit)** / **+37 pp mAP@10** lift over the frozen-CLIP baseline. The headline is the mean $\pm$ std over four independent fine-tuning runs (seeds 33, 83, 527, 588), each evaluated on the full 14,218-query set against a fresh per-seed gallery index. The seed-83 weights (top of every metric column) are saved as `artifacts/clip_finetuned_best.pt` and are the recommended single-model checkpoint for the demo and batch eval.

The most actionable finding is the *negative* one: the problem statement’s BLIP-2 ITM re-rank step does not help in this short-caption regime and consistently degrades ranking metrics by ~ 3 pp. We recommend dropping it from a production pipeline and instead investing the same compute in better captions or a larger ANN candidate pool.

Appendix A — Reproducibility

All numbers in this report come from `artifacts/eval_*.json`, all locally re-runnable end-to-end with `rerun_all_metrics.ipynb`: it rebuilds the per-seed $\alpha=0.7$ gallery indices, runs the full 14,218-query evaluation per condition, and prints the 4-seed aggregate inline.

The recommended headline weight is `artifacts/clip_finetuned_best.pt` (seed-83 checkpoint). The three sibling-seed checkpoints (`clip_finetunedFinal_{33,527,588}.pt`) reproduce the mean $\pm$ std numbers. Per-seed source notebooks: `notebooks/{33,83,527,588}_roll_clip_ft.ipynb`. Commit hashes for the code: see `git log` on the GitHub repository.